



Getting started with the REST API

Learn how to use the GitHub REST API.

GitHub CLI curl JavaScript

Copy as Markdown ▾

In this article

Introduction

About requests to the REST API

Making a request

Using the response

Rate limiting

Next steps

Introduction

This article describes how to use the GitHub REST API with GitHub CLI, `curl`, or JavaScript. For a quickstart guide, see [Quickstart for GitHub REST API](#).

About requests to the REST API

This section describes the elements that make up an API request:

- [HTTP method](#)
- [Path](#)
- [Headers](#)
- [Media types](#)
- [Authentication](#)

- [Parameters](#)

Every request to the REST API includes an HTTP method and a path. Depending on the REST API endpoint, you might also need to specify request headers, authentication information, query parameters, or body parameters.

The REST API reference documentation describes the HTTP method, path, and parameters for every endpoint. It also displays example requests and responses for each endpoint. For more information, see the [REST reference documentation](#).

HTTP method

The HTTP method of an endpoint defines the type of action it performs on a given resource. Some common HTTP methods are `GET`, `POST`, `DELETE`, and `PATCH`. The REST API reference documentation provides the HTTP method for every endpoint.

For example, the HTTP method for the ["List repository issues" endpoint](#) is `GET`.

Where possible, the GitHub REST API strives to use an appropriate HTTP method for each action.

- `GET` : Used for retrieving resources.
- `POST` : Used for creating resources.
- `PATCH` : Used for updating properties of resources.
- `PUT` : Used for replacing resources or collections of resources.
- `DELETE` : Used for deleting resources.

Path

Each endpoint has a path. The REST API reference documentation gives the path for every endpoint. For example, the path for the ["List repository issues" endpoint](#) is `/repos/{owner}/{repo}/issues`.

The curly brackets `{ }` in a path denote path parameters that you need to specify. Path parameters modify the endpoint path and are required in your request. For example, the path parameters for the ["List repository issues" endpoint](#) are `{owner}` and `{repo}`. To use this path in your API request, replace `{repo}` with the name of the repository where you would like to request a list of issues, and replace `{owner}` with the name of the account that owns the repository.

Headers

Headers provide extra information about the request and the desired response. Following are some examples of headers that you can use in your requests to the GitHub REST API. For an

example of a request that uses headers, see [Making a request](#).

Accept

Most GitHub REST API endpoints specify that you should pass an `Accept` header with a value of `application/vnd.github+json`. The value of the `Accept` header is a media type. For more information about media types, see [Media types](#).

X-GitHub-API-Version

You should use this header to specify a version of the REST API to use for your request. For more information, see [API Versions](#).

User-Agent

All API requests must include a valid `User-Agent` header. The `User-Agent` header identifies the user or application that is making the request.

By default, GitHub CLI sends a valid `User-Agent` header. However, GitHub recommends using your GitHub username, or the name of your application, for the `User-Agent` header value. This allows GitHub to contact you if there are problems.

The following is an example `User-Agent` for an app named `Awesome-Octocat-App`:

```
User-Agent: Awesome-Octocat-App
```

Requests with no `User-Agent` header will be rejected. If you provide an invalid `User-Agent` header, you will receive a `403 Forbidden` response.

Media types

You can specify one or more media types by adding them to the `Accept` header of your request. For more information about the `Accept` header, see [Accept](#).

Media types specify the format of the data you want to consume from the API. Media types are specific to resources, allowing them to change independently and support formats that other resources don't. The documentation for each GitHub REST API endpoint will describe the media types that it supports. For more information, see the [GitHub REST API documentation](#).

The most common media types supported by the GitHub REST API are `application/vnd.github+json` and `application/json`.

There are custom media types that you can use with some endpoints. For example, the REST API to manage [commits](#) and [pull requests](#) support the media types `diff`, `patch`, and `sha`. The media types `full`, `raw`, `text`, or `html` are used by some other endpoints.

All custom media types for GitHub look like this: `application/vnd.github.PARAM+json`, where `PARAM` is the name of the media type. For example, to specify the `raw` media type, you would use `application/vnd.github.raw+json`.

For an example of a request that uses media types, see [Making a request](#).

Authentication

Many endpoints require authentication or return additional information if you are authenticated. Additionally, you can make more requests per hour when you are authenticated.

Although some REST API endpoints are accessible without authentication, GitHub CLI requires you to authenticate before you can use the `api` subcommand to make an API request. Use the `auth login` subcommand to authenticate to GitHub. For more information, see [Making a request](#).

Parameters

Many API methods require or allow you to send additional information in parameters in your request. There are a few different types of parameters: Path parameters, body parameters, and query parameters.

Path parameters

Path parameters modify the endpoint path. These parameters are required in your request. For more information, see [Path](#).

Body parameters

Body parameters allow you to pass additional data to the API. These parameters can be optional or required, depending on the endpoint. For example, a body parameter may allow you to specify an issue title when creating a new issue, or specify certain settings when enabling or disabling a feature. The documentation for each GitHub REST API endpoint will describe the body parameters that it supports. For more information, see the [GitHub REST API documentation](#).

For example, the ["Create an issue" endpoint](#) requires that you specify a title for the new issue in your request. It also allows you to optionally specify other information, such as text to put in

the issue body, users to assign to the new issue, or labels to apply to the new issue. For an example of a request that uses body parameters, see [Making a request](#).

You must authenticate your request to pass body parameters. For more information, see [Authentication](#).

Query parameters

Query parameters allow you to control what data is returned for a request. These parameters are usually optional. The documentation for each GitHub REST API endpoint will describe any query parameters that it supports. For more information, see the [GitHub REST API documentation](#).

For example, the ["List public events" endpoint](#) returns thirty issues by default. You can use the `per_page` query parameter to return two issues instead of 30. You can use the `page` query parameter to fetch only the first page of results. For an example of a request that uses query parameters, see [Making a request](#).

Making a request

This section demonstrates how to make an authenticated request to the GitHub REST API using GitHub CLI.

1. Setup

Install GitHub CLI on macOS, Windows, or Linux. For more information, see [Installation](#) in the GitHub CLI repository.

2. Authenticate

- 1 To authenticate to GitHub, run the following command from your terminal.

```
gh auth login
```

You can use the `--scopes` option to specify what scopes you want. If you want to authenticate with a token that you created, you can use the `--with-token` option. For more information, see the [GitHub CLI auth login documentation](#).

- 2 Select where you want to authenticate to:
 - If you access GitHub at GitHub.com, select **GitHub.com**.

- If you access GitHub at a different domain, select **Other**, then enter your hostname (for example: `octocorp.ghe.com`).

3 Follow the rest of the on-screen prompts.

GitHub CLI automatically stores your Git credentials for you when you choose HTTPS as your preferred protocol for Git operations and answer "yes" to the prompt asking if you would like to authenticate to Git with your GitHub credentials. This can be useful as it allows you to use Git commands like `git push` and `git pull` without needing to set up a separate credential manager or use SSH.

3. Choose an endpoint for your request

- 1 Choose an endpoint to make a request to. You can explore GitHub's [REST API documentation](#) to discover endpoints that you can use to interact with GitHub.
- 2 Identify the HTTP method and path of the endpoint. You will send these with your request. For more information, see [HTTP method](#) and [Path](#).

For example, the ["Create an issue" endpoint](#) uses the HTTP method `POST` and the path `/repos/{owner}/{repo}/issues` .

- 3 Identify any required path parameters. Required path parameters appear in curly brackets `{}` in the path of the endpoint. Replace each parameter placeholder with the desired value. For more information, see [Path](#).

For example, the ["Create an issue" endpoint](#) uses the path `/repos/{owner}/{repo}/issues` , and the path parameters are `{owner}` and `{repo}` . To use this path in your API request, replace `{repo}` with the name of the repository where you would like to create a new issue, and replace `{owner}` with the name of the account that owns the repository.

4. Make a request with GitHub CLI

Use the GitHub CLI `api` subcommand to make your API request. For more information, see the [GitHub CLI `api` documentation](#).

In your request, specify the following options and values:

- **--method** followed by the HTTP method and the path of the endpoint. For more information, see [HTTP method](#) and [Path](#).
- **--header:**
 - **Accept** : Pass the media type in an `Accept` header. To pass multiple media types in an `Accept` header, separate the media types with a comma: `Accept:`

`application/vnd.github+json`, `application/vnd.github.diff` . For more information, see [Accept](#) and [Media types](#).

- **X-GitHub-API-Version** : Pass the API version in a `X-GitHub-API-Version` header. For more information, see [X-GitHub-API-Version](#) .

- `-f` or `-F` followed by any body parameters or query parameters in `key=value` format. Use the `-F` option to pass a parameter that is a number, Boolean, or null. Use the `-f` option to pass string parameters.

Some endpoints use query parameters that are arrays. To send an array in the query string, use the query parameter once per array item, and append `[]` after the query parameter name. For example, to provide an array of two repository IDs, use `-f repository_ids[]=REPOSITORY_A_ID -f repository_ids[]=REPOSITORY_B_ID` .

If you do not need to specify any body parameters or query parameters in your request, omit this option. For more information, see [Body parameters](#) and [Query parameters](#). For examples, see [Example request using body parameters](#) and [Example request using query parameters](#).

Example request

The following example request uses the ["Get Octocat" endpoint](#) to return the octocat as ASCII art.

Shell



```
gh api --method GET /octocat \  
--header 'Accept: application/vnd.github+json' \  
--header "X-GitHub-API-Version: 2022-11-28"
```

Example request using query parameters

The ["List public events" endpoint](#) returns thirty issues by default. The following example uses the `per_page` query parameter to return two issues instead of 30, and the `page` query parameter to fetch only the first page of results.

Shell



```
gh api --method GET /events -F per_page=2 -F page=1  
--header 'Accept: application/vnd.github+json' \  

```

Example request using body parameters

The following example uses the ["Create an issue" endpoint](#) to create a new issue in the octocat/Spoon-Knife repository. In the response, find the `html_url` of your issue, and navigate to your issue in the browser.

Shell



```
gh api --method POST /repos/octocat/Spoon-Knife/issues \
--header "Accept: application/vnd.github+json" \
--header "X-GitHub-API-Version: 2022-11-28" \
-f title='Created with the REST API' \
-f body='This is a test issue created by the REST API' \
```

Using the response

After you make a request, the API will return the response status code, response headers, and potentially a response body.

About the response code and headers

Every request will return an HTTP status code that indicates the success of the response. For more information about response codes, see [the MDN HTTP response status code documentation](#).

Additionally, the response will include headers that give more details about the response. Headers that start with `X-` or `x-` are custom to GitHub. For example, the `x-ratelimit-remaining` and `x-ratelimit-reset` headers tell you how many requests you can make in a time period.

To view the status code and headers, use the `--include` or `--i` option when you send your request.

For example, this request gets a list of issues in the octocat/Spoon-Knife repository:

```
gh api \
--header 'Accept: application/vnd.github+json' \
--method GET /repos/octocat/Spoon-Knife/issues \
-F per_page=2 --include
```

And it returns a response code and headers that look something like this:

```
HTTP/2.0 200 OK
Access-Control-Allow-Origin: *
Access-Control-Expose-Headers: ETag, Link, Location, Retry-After, X-RateLimit-
```

```
Limit, X-RateLimit-Remaining, X-RateLimit-Used, X-RateLimit-Resource, X-
RateLimit-Reset, X-0Auth-Scopes, X-Accepted-0Auth-Scopes, X-Poll-Interval, X-
GitHub-Media-Type, X-GitHub-SSO, X-GitHub-Request-Id, Deprecation, Sunset
Cache-Control: private, max-age=60, s-maxage=60
Content-Security-Policy: default-src 'none'
Content-Type: application/json; charset=utf-8
Date: Thu, 04 Aug 2022 19:56:41 GMT
Etag: W/"a63dfbcfdb73621e9d2e89551edcf9856731ced534bd7f1e114a5da1f5f73418"
Link: <https://api.github.com/repositories/1300192/issues?per_page=1&page=2>;
rel="next", <https://api.github.com/repositories/1300192/issues?
per_page=1&page=14817>; rel="last"
Referrer-Policy: origin-when-cross-origin, strict-origin-when-cross-origin
Server: GitHub.com
Strict-Transport-Security: max-age=31536000; includeSubdomains; preload
Vary: Accept, Authorization, Cookie, Accept-Encoding, Accept, X-Requested-With
X-Accepted-0auth-Scopes: repo
X-Content-Type-Options: nosniff
X-Frame-Options: deny
X-Github-API-Version-Selected: 2022-08-09
X-Github-Media-Type: github.v3; format=json
X-Github-Request-Id: 1C73:26D4:E2E500:1EF78F4:62EC2479
X-0auth-Client-Id: 178c6fc778ccc68e1d6a
X-0auth-Scopes: gist, read:org, repo, workflow
X-Ratelimit-Limit: 15000
X-Ratelimit-Remaining: 14996
X-Ratelimit-Reset: 1659645499
X-Ratelimit-Resource: core
X-Ratelimit-Used: 4
X-Xss-Protection: 0
```

In this example, the response code is `200`, which indicates a successful request.

About the response body

Many endpoints will return a response body. Unless otherwise specified, the response body is in JSON format. Blank fields are included as `null` instead of being omitted. All timestamps return in UTC time, ISO 8601 format: `YYYY-MM-DDTHH:MM:SSZ`.

Unlike the GraphQL API where you specify what information you want, the REST API typically returns more information than you need. If desired, you can parse the response to pull out specific pieces of information.

For example, you can use `>` to redirect the response to a file. In the following example, replace `REPO-OWNER` with the name of the account that owns the repository, and `REPO-NAME` with the name of the repository.

Shell



```
gh api \
--header 'Accept: application/vnd.github+json' \
--method GET /repos/REPO-OWNER/REPO-NAME/issues \
-F per_page=2 > data.json
```

Then you can use `jq` to get the title and author ID of each issue:

Shell



```
jq '.[0] | {title: .title, authorID: .user.id}' data.json
```

The previous two commands return something like:

```
{
  "title": "Update index.html",
  "authorID": 10701255
}
{
  "title": "Edit index file",
  "authorID": 53709285
}
```

For more information about `jq`, see [the jq documentation](#).

Detailed versus summary representations

A response can include all attributes for a resource or only a subset of attributes, depending on whether you fetch an individual resource or a list of resources.

- When you fetch an *individual resource*, like a specific repository, the response will typically include all attributes for that resource. This is the "detailed" representation of the resource.
- When you fetch a *list of resources*, like a list of multiple repositories, the response will only include a subset of the attributes for each resource. This is the "summary" representation of the resource.

Note that authorization sometimes influences the amount of detail included in a representation.

The reason for this is because some attributes are computationally expensive for the API to provide, so GitHub excludes those attributes from the summary representation. To obtain those attributes, you can fetch the detailed representation.

The documentation provides an example response for each API method. The example response illustrates all attributes that are returned by that method.

Hypermedia

All resources may have one or more `*_url` properties linking to other resources. These are meant to provide explicit URLs so that proper API clients don't need to construct URLs on their own. It is highly recommended that API clients use these. Doing so will make future upgrades of the API easier for developers. All URLs are expected to be proper [RFC 6570](#) URI templates.

You can then expand these templates using something like the [uri_template](#) gem:

```
>> tmpl = URITemplate.new('/notifications {?since,all,participating}')
>> tmpl.expand
=> "/notifications"

>> tmpl.expand all: 1
=> "/notifications?all=1"

>> tmpl.expand all: 1, participating: 1
=> "/notifications?all=1&participating=1"
```

Rate limiting

The GitHub REST API limits the number of requests you can make within a given time period. For more information about rate limits and how to check your current rate limit status, see [Rate limits for the REST API](#).

Next steps

This article demonstrated how to list and create issues in a repository. For more practice, try to comment on an issue, edit the title of an issue, or close an issue. For more information, see the ["Create an issue comment" endpoint](#) and the ["Update an issue" endpoint](#).

For more information about other endpoints that you can use, see the [REST reference documentation](#).

Legal

© 2026 GitHub, Inc. [Terms](#) [Privacy](#) [Status](#) [Pricing](#) [Expert services](#) [Blog](#)